The Universal Language: A Characteristica Universalis for AI

Greg Coppola, PhD
coppola.ai

February 23, 2026

Abstract

In previous work we introduced the Logical Bayesian Network (Coppola, 2024), a graphical model that unifies logical and probabilistic reasoning via AND/OR/NEG boolean factors with belief propagation, and demonstrated a pipeline from natural language to logical inference via a typed slot grammar with LLM-assisted disambiguation (Coppola, 2026). Those papers contributed the *machine* (inference) and the *bridge* (syntax). This paper contributes the *meaning*: we argue that the typed logical language underlying both systems is a realization of Leibniz's *characteristica universalis*—a universal formal language sufficient for expressing any proposition expressible in natural language—and that the Logical Bayesian Network is a realization of his *calculus ratiocinator*—a mechanical procedure for reasoning over that language.

This thesis has four direct consequences. First, **hallucination** is structurally impossible: the calculus produces conclusions only through explicit derivations, so ungrounded output is architecturally excluded. Second, **interpretability** is free: the factor graph *is* the explanation, built from human-readable rules over typed propositions. Third, **reinforcement learning from verification feedback** (RLVF) becomes possible: the characteristica and calculus provide a mechanical verifier for reasoning, analogous to compilers for code, enabling scalable RL training without human reward signals. Fourth, **alignment** reduces to verification: if both human intent and AI output are expressed in the characteristica, misalignment is a detectable logical error rather than a philosophical puzzle.

We support the thesis with three independent arguments for the uniqueness of the language (historical convergence, parsimony, completeness), explain why large language models dissolve the annotation bottleneck that historically prevented formal semantics from scaling, present evidence that the transformer architecture independently converges on the same computational structure, and demonstrate the characteristica's expressiveness by formalizing the thesis arguments themselves within the system. All code and examples are open source at `https://github.com/gregorycoppola/world`.

1 Introduction

In February 2024 we introduced the Quantified Boolean Bayesian Network (Coppola, 2024), a graphical model that decomposes probabilistic inference over logical propositions into AND and OR boolean factors with iterative belief propagation. In February 2026 we extended this system with NEG factors for contrapositive reasoning, a typed logical language with three tiers of expressiveness, a typed slot grammar for deterministic compilation from natural language, and an LLM-assisted disambiguation pipeline (Coppola, 2026). Together, these papers built a complete pipeline from natural language text to probabilistic logical inference.

This paper asks: what is the significance of the formal language at the center of that pipeline?

Our answer is that the typed predicate language—entities with types, predicates with labeled roles, quantified Horn clauses, negation, propositions as arguments, and lambda abstraction—is not merely a useful engineering formalism. It is a computational realization of Leibniz's *characteristica universalis*: a universal language of concepts sufficient for expressing any proposition expressible in natural language. And the Logical Bayesian Network—belief propagation over AND/OR/NEG factor graphs parameterized by weighted Horn clauses—is a realization of his *calculus ratiocinator*: a mechanical procedure that reasons over the characteristica, extended from certainty to degrees of belief.

The pieces are not new individually. Typed predicate logic is due to Frege (1879). Natural deduction is due to Gentzen (1934) and Prawitz (1965). The connection to natural language is due to Montague (1973) and Steedman (1996). Probabilistic graphical models are due to Pearl (1988). What is new is the claim that these pieces compose into a universal language for AI—and that this composition has immediate, practical consequences for the four largest open problems in the field.

1.1 Four Consequences

The universal language thesis yields four corollaries:

1. **Hallucination is structurally impossible.** If all reasoning must pass through the calculus, and the calculus only produces conclusions with derivations through the factor graph, then ungrounded output is architecturally excluded.

2. **Interpretability is free.** The factor graph *is* the explanation. Every conclusion traces through named rules, typed propositions, and explicit causal chains. There is nothing to interpret because the reasoning is already in human-readable form.

3. **RLVF becomes possible.** The characteristica and calculus provide a mechanical verifier for reasoning tasks. LLMs learned to code because code has a compiler. Reasoning now has one too. Reinforcement learning from verification feedback scales with compute, not humans.

4. **Alignment becomes verification.** If both human intent and AI output are expressed in the characteristica, misalignment is a detectable logical error—a failed derivation—not a philosophical puzzle.

These are not four separate solutions. They are the same insight applied four ways. The characteristica grounds output (hallucination). The proof tree is readable (interpretability). The verification is a reward signal (RLVF). The shared language makes misalignment detectable (alignment). One formal language, four payoffs.

1.2 Contributions

This is the third paper in a series. The three papers correspond to three classical areas of natural language processing:

	Inference	Semantics	Syntax
Paper 1 (2024)	★★★	★★	—
Paper 2 (2026)	★★	★★	★★★
Paper 3 (this)	★	★★★	★

Paper 1 built the inference engine and sketched the predicate language. Paper 2 built the parsing pipeline and defined the language formally. This paper argues that the language is *the* language—the universal one—and draws the consequences.

1.3 Roadmap

Section 3 recaps the prior work. Section 4 states the universal language thesis. Section 5 argues why this particular language is the right one. Sections 6–9 develop the four corollaries. Section 10 explains why this is possible now. Section 11 presents evidence from the transformer architecture. Section 12 extends the characteristica to software engineering. Section 13 demonstrates the system by formalizing the thesis arguments themselves. Section 14 discusses related work. Section 15 concludes.

2 Prior Work: The Machine and the Bridge

We briefly summarize the two preceding papers. The reader is referred to Coppola (2024) and Coppola (2026) for full details. This section establishes what exists so that the present paper can focus on what it means.

2.1 Paper 1: The Logical Bayesian Network (2024)

Coppola (2024) introduced the Quantified Boolean Bayesian Network (QBBN), now called the Logical Bayesian Network (LBN): a Bayesian network in which all factors are decomposed into boolean gates over typed propositions.

The key contributions were:

Boolean decomposition. All factors split into deterministic AND gates (conjunction) and learned OR gates (weighted disjunction via noisy-OR). This produces a bipartite graph alternating between proposition nodes and group nodes, enabling efficient message passing.

Belief propagation. The boolean decomposition makes iterative belief propagation tractable at $O(N \cdot 2^n)$ per pass, where N is the number of network variables and n bounds the fan-in of any gate. Empirical experiments confirmed convergence on all tested structures.

Key-value calculus. First-order logic was reformulated from positional arguments to role-labeled key-value pairs: `wrote(arg0: Shakespeare, arg1: Macbeth)` rather than `wrote(Shakespeare, Macbeth)`. This minimizes the distance between dependency parse structure and logical representation.

Quantification as parameter sharing. Universal quantification was realized as weight sharing across grounded instances, enabling "infinite use of finite means."

The 2024 paper left two gaps: no negation or contrapositive reasoning, and no parser from natural language to logical forms.

2.2 Paper 2: Statistical Parsing for Logical Information Retrieval (2026)

Coppola (2026) addressed both gaps and contributed across all three areas.

Inference. NEG factors were introduced, linking each proposition to its negation and enforcing $P(x) + P(\neg x) = 1$. Combined with bidirectional proposition graph construction, this completed the implementation of Prawitz's simple elimination rules—both modus ponens and modus tollens—within a single belief propagation loop. The inference engine passed 44 out of 44 test cases spanning 22 reasoning patterns.

Semantics. A typed logical language was defined with three tiers of expressiveness following Prawitz (1965): first-order quantification over entities, propositions as arguments via a sentential type, and a designed extension to predicate quantification via lambda abstraction. The paper claimed these three tiers are sufficient for natural language semantics.

Syntax. A typed slot grammar was presented that deterministically compiles disambiguated input to logical form, achieving 33 out of 33 sentences with zero ambiguity. LLM-assisted preprocessing achieved 95% accuracy on PP attachment disambiguation, while LLMs failed at direct structured parsing (12.4% UAS), confirming that grammars are necessary for exact structure and LLMs are necessary for disambiguation. The resulting pipeline is: LLM preprocesses → grammar parses → LLM reranks → LBN infers.

2.3 What Both Papers Left Open

Neither paper addressed the *significance* of the typed logical language. It was presented as engineering—"here is the language our model operates over," "here is what the grammar needs to target." The language was never connected to Leibniz, to Chomsky's universal grammar, or to the philosophical tradition of formal semantics. The sufficiency claim was made but buried. The consequences for hallucination, interpretability, RLVF, and alignment were not drawn. This paper draws them.

3 The Universal Language Thesis

3.1 The Characteristica Universalis

In 1685, Leibniz proposed the *characteristica universalis*: a universal formal language in which all human knowledge could be expressed, and a *calculus ratiocinator*: a mechanical procedure by which disputes could be resolved through calculation rather than argument. "Let us calculate," he wrote, imagining that disagreements could be settled by reducing them to formal computation.

We claim that the typed logical language developed in Coppola (2024, 2026) is a realization of the characteristica, and that the Logical Bayesian Network is a realization of the calculus.

3.2 The Language

The characteristica consists of:

Entities. Atomic objects identified by strings and organized by types: `socrates : e`, `france : country`, `macbeth : book`.

Predicates with labeled roles. Each predicate is defined by a function name and a set of typed roles: `trust{agent: e, patient: e}`, `mortal{theme: e}`. A fully grounded predicate is a proposition with a truth value and a probability.

Quantified Horn clauses. Universally quantified implications with modal weights: `always [x:e]: man(theme: x) -> mortal(theme: x)`. The quantifier words—*always*, *usually*, *sometimes*—map to weights parameterizing the OR factor, bridging proof theory and probability.

Negation. Ground facts and rule conclusions can be negated, enforced by NEG factors in the factor graph.

Three tiers of expressiveness following Prawitz (1965): first-order quantification over entities (Tier 1), propositions as arguments via a sentential type (Tier 2), and predicate quantification via lambda abstraction (Tier 3).

3.3 The Calculus

The calculus ratiocinator is the Logical Bayesian Network: belief propagation over a factor graph composed of AND, OR, and NEG factors. Given a knowledge base of ground facts and quantified rules, the LBN grounds the rules, builds a factor graph via bidirectional expansion from the query, and runs iterative belief propagation to compute posterior probabilities.

The calculus implements the simple elimination rules of Prawitz's natural deduction: modus ponens (forward π messages through OR factors), modus tollens (backward λ messages through NEG factors), conjunction introduction and elimination (AND factors), and universal instantiation (grounding). This is the forward fragment—linear time, guaranteed termination, tractable inference.

Leibniz's calculus was deterministic: conclusions follow or they do not. Our extension is probabilistic: conclusions follow with degrees of belief, parameterized by modal quantifiers. This is the unification of logic and statistics that the first paper contributed—Leibniz's dream extended from certainty to probability.

3.4 The Sufficiency Claim

We claim the characteristica is sufficient for expressing any proposition expressible in natural language. This follows from three observations:

First, Prawitz (1965) proved that the rules of natural deduction are complete and consistent for first-order logic: every true statement has a proof, and every provable statement is true.

Second, first-order logic (with extensions to second-order and modal logic, which Prawitz also treats) is accepted in the philosophy of science as sufficient for expressing mathematics and science.

Third, the three tiers of the characteristica—quantification over entities, propositions as arguments, and predicate quantification—correspond to Prawitz's three levels of logical expressiveness. Together they cover predication, classification, quantified rules, conjunction, disjunction, transitivity, symmetry, causation, modality, propositional attitudes, and compound predicates.

The remaining work is lexical, not logical: determining which words map to which predicates and types. The logical machinery is sufficient. The dictionary is not yet complete.

4 Why This Language

One might object that many formal languages exist—why is this one special? We offer three independent arguments that the characteristica is not an arbitrary choice but the uniquely correct formal language for grounding reasoning.

4.1 Convergence

Every independent attempt to formalize reasoning—across 2400 years, multiple cultures, and radically different starting assumptions—converges on the same language and the same rules:

Who	When	Language	Rules
Aristotle	350 BC	Subject-predicate structure	Syllogistic inference
Pāṇini	∼400 BC	Typed grammatical categories	Rule-based derivation
Leibniz	1685	Atomic concepts + combination	Calculus ratiocinator
Frege	1879	Typed predicate logic	Modus ponens, quantifiers
Gentzen	1934	Natural deduction	Introduction/elimination
Prawitz	1965	Three tiers of natural deduction	Twelve rules for FOL
Montague	1973	Typed lambda calculus	Compositional semantics

Both the language (typed entities, predicates with argument structure, quantification) and the rules (introduction and elimination of connectives) are convergent. When every civilization that attempts to formalize reasoning arrives at the same structure, the structure is a feature of reasoning itself—discovered, not invented.

4.2 Parsimony

The formal system is minimal in three senses:

Computationally: the forward fragment is linear time, decidable, with guaranteed termination—the cheapest calculus that supports useful inference.

Representationally: typed predicates with role labels are the shortest encoding of relational knowledge—less redundant than natural language, less opaque than embeddings.

Conceptually: three primitives (entities, predicates, rules) and twelve inference rules. The system fits on one page. Remove any piece and you lose expressiveness or completeness. Add anything and you gain complexity without coverage.

Three independent measures of simplicity pointing to the same system is evidence that the system is fundamental.

4.3 Completeness

Prawitz's twelve rules of natural deduction are provably complete and consistent for first-order logic: every true statement has a proof, and every provable statement is true. First-order logic is Turing-complete: it can express any computable claim.

The calculus ratiocinator is not a partial solution. It is the complete engine for verification. We start with the tractable forward fragment, but the framework encompasses all of reasoning—query, planning, full theorem proving—with each fragment at a natural boundary in computational complexity.

4.4 Not Lean

One might ask why a new formal language is needed when proof assistants like Lean and Coq already exist. The answer is that Lean verifies *mathematics*; the characteristica verifies *reasoning expressed in natural language*. Lean's type theory is designed for mathematical proof, not for representing that "Jack trusts Jill" or "all requests require authentication." The characteristica operates at the level of natural language semantics—typed predicates with role labels that map directly to dependency parse structure. It is the universal grammar of thought, not a proof assistant for theorems.

5 Hallucination

5.1 The Problem

Large language models produce fluent, confident text that is factually wrong. They invent citations, attribute false quotes, describe events that never happened—all with the same tone as when they are correct. This is the most widely reported limitation of LLMs and the primary barrier to their use in high-stakes applications.

5.2 Why It Happens

An LLM is a function from token sequences to probability distributions over next tokens. It lacks all three components of verified reasoning:

No *characteristica*: no typed propositions, no discrete truth-evaluable representations. Only continuous embeddings with no inspectable logical structure.

No *calculus ratiocinator*: no inference rules. No modus ponens, no universal instantiation, no derivation. Output is sampled from a distribution, not derived from premises.

No *grounding*: no knowledge base. The model cannot check truth, detect contradictions, or distinguish knowledge from ignorance.

Without the language, the rules, and the grounding, an LLM cannot verify its own output. It can only generate text that resembles the output of reasoning.

5.3 Why Scaling Does Not Fix It

Larger models hallucinate less frequently. But the improvement is asymptotic, not eliminative. Scaling adds parameters—more refined distributions over tokens. It does not add a characteristica, a calculus, or a grounding. The architecture lacks all three at any scale.

5.4 The Solution

Ground the output in the characteristica. Check it with the calculus.

Parse the LLM's natural language output into typed logical forms. Bind them to a knowledge base. Apply the inference rules. The proof tree shows which facts combined by which rules to produce the conclusion.

Does the logical form type-check? The characteristica is well-formed. Does it follow from the knowledge base? The calculus derives it. Is it consistent with known facts? The grounding constrains it.

Hallucination becomes a detectable, correctable error rather than a silent failure. The LLM proposes. The characteristica grounds. The calculus checks. This is not a post-hoc fix applied to the LLM's output. It is an architecture in which hallucination cannot occur—because there is no mechanism to produce a conclusion without a derivation.

6 Interpretability

6.1 The Problem

Neural networks are opaque. A billion parameters cannot be inspected to understand why the model produced a particular output. The interpretability research program attempts to make the internals readable—through attention visualization, probing classifiers, mechanistic interpretability, and related techniques.

6.2 The Reframe

Interpretability means expressing reasoning in a language humans understand, with rules humans follow, grounded in facts humans can inspect. This requires all three components:

The *characteristica* provides a readable language: typed predicates with role labels map directly to natural language. `trust(agent: jack, patient: jill)`—you can read it.

The *calculus ratiocinator* provides intuitive rules: modus ponens, universal instantiation, forward chaining. A proof tree built from these rules is a proof tree humans can follow—because these are the rules humans already reason with.

The *grounding* provides explicit facts: the knowledge base lists what is assumed true. You can inspect it, challenge it, update it. Nothing is hidden.

A proof tree connects all three: "Given *these facts*, applying *these rules*, expressed in *these terms*, *this conclusion* follows." That is what understanding an inference means.

6.3 The Deeper Claim

If the universal language thesis is correct, then humans reason in the characteristica and calculus implicitly—as the structure and logic of thought. LLMs learn both implicitly from training data that encodes them. The formal language and rules make both explicit.

An LLM that has learned to reason has learned something that corresponds to typed predicates and the rules of natural deduction—encoded in distributed representations rather than discrete symbols. Interpretability, in this view, is translating that distributed encoding back into the symbolic language and rules that humans read.

The symbols and rules are not a workaround for opacity. They are the medium of understanding itself. Interpretability is not an add-on. It is a property of reasoning in the characteristica—free by construction.

7 Reinforcement Learning from Verification Feedback

7.1 The Observation

LLMs learned to code because code has a verifier. You write Python, the compiler checks it, tests pass or fail. That binary signal is the reward. Scale it with reinforcement learning and you get systems that are genuinely good at programming.

LLMs have not learned to *reason* because reasoning has no verifier. You write an argument in English, and no mechanical process checks it. Human evaluation is slow, expensive, inconsistent, and foolable by fluent nonsense. There is no compiler for reasoning.

The characteristica and calculus ratiocinator are the missing compiler.

7.2 The Pattern

Domain	LLM generates	Verifier	Reward
Code	Python / Rust	Compiler + tests	Pass / fail
Math	Lean / Coq	Type checker	Proof checks or not
Reasoning	Typed logical forms	Calculus ratiocinator	Inference succeeds or not

The pattern is identical. The only thing missing for reasoning was the verifier. Now it exists.

7.3 RLVF vs. RLHF

RLHF trains models to produce output that *looks* correct to humans. Humans are fooled by fluent, confident nonsense. The model learns to produce fluent, confident nonsense. This is why RLHF-trained models still hallucinate—they have been optimized for the appearance of correctness.

RLVF trains models to produce output that *is* correct, as verified by the calculus. The model cannot fool the verifier. A logical form either checks or it does not.

The failure modes are revealing. RLHF failure: "I am confident this is true" (it is not)—confident hallucination. RLVF failure: "I cannot produce a valid proof"—honest inability. Honest inability is preferable to confident hallucination.

7.4 The Flywheels

The verification signal enables several self-reinforcing loops:

Training loop. LLM generates logical forms → calculus verifies → reward signal trains better LLM → better logical forms. This scales with compute, not humans.

KB bootstrapping. The same LLM generates lexicon entries, predicates, rules, entire knowledge bases. The calculus checks consistency. Valid entries expand the KB. Larger KB enables more training examples. More examples improve the LLM.

Bidirectional training. Train the LLM in both directions—natural language to logical form (parsing) and logical form to natural language (generation). Both directions have verifiable signals. Round-trip consistency is a self-supervised signal requiring no human annotation.

Verifiable chain-of-thought. Instead of externalizing reasoning in unverifiable English, the LLM externalizes it in the characteristica. Every step is checkable. This is chain-of-thought with a proof tree—not text that looks like reasoning, but reasoning that is reasoning.

7.5 Self-Generating Training Data

In code RLVF, someone must write the test cases. In reasoning RLVF, the verifier generates its own training data. Start with a knowledge base. The calculus enumerates all conclusions derivable from it. Each (KB, conclusion) pair is a training example. The training data grows with the KB. The KB grows with the LLM. Both grow with compute. No human writes test cases.

8 Alignment

8.1 The Problem

AI alignment is usually framed as: how do we make AI do what we want? This framing is intractable because "what we want" is expressed in natural language, which is ambiguous, and the AI's internal process is opaque. You cannot check whether an opaque system satisfies an ambiguous specification.

8.2 The Reframe

If human and AI share a formal language with verification, alignment becomes a different kind of problem: does the AI's formal output match the human's formal intent? This is a query in the characteristica, verifiable by the calculus.

The pipeline works as follows. The human expresses intent in natural language. The pipeline parses it into the characteristica. The AI produces output, also parsed into the characteristica.

The calculus checks: does the output satisfy the intent? Do the conclusions follow from the stated premises? Are the constraints respected? Is the output consistent with the knowledge base?

Misalignment becomes a detectable, diagnosable error—not a philosophical puzzle. The proof tree shows *where* the AI's reasoning diverged from the human's intent. The human can inspect it, point to the step that went wrong, and correct it.

8.3 Alignment as a Property of the Representation

Alignment is usually treated as separate from capability. But if the AI reasons in the characteristica:

Capability = can the AI produce valid logical forms that solve the task?

Alignment = do those logical forms match the human's stated intent?

Both are queries over the same formal language. Both are checked by the same calculus. Alignment is not a separate safety layer bolted onto capability. It is a property of the shared representation—verifiable by the same machinery that verifies reasoning.

8.4 Alignment Scales with Verification

RLHF alignment scales with human reviewers. There are not enough. RLVF alignment scales with compute. The calculus checks whether the AI's output satisfies the formal specification of intent. This check is mechanical, deterministic, and instantaneous. As the characteristica covers more of natural language reasoning, alignment coverage grows automatically.

Alignment becomes verification. Verification scales.

9 Why Now: The Annotation Bottleneck

9.1 The Problem

The characteristica has been known since Frege (1879). The calculus since Gentzen (1934). Montague (1973) connected them to natural language. If the universal language thesis is correct, why has it not been realized before?

The answer is the annotation bottleneck. Formal methods have always worked in principle. They failed in practice because someone had to write the characteristica by hand.

It killed formal NLP: Montague semantics works, but annotating every sentence with a logical form does not scale. It limited formal verification: TLA+ works, but writing specifications for every system does not scale. The Penn Treebank took years to construct and covered only syntax, with no formal semantics. FrameNet, PropBank, and the Abstract Meaning Representation project all hit the same wall. Human annotators are slow, expensive, inconsistent, and finite. The formal approach scaled linearly with human effort, which is to say it did not scale at all.

9.2 The Solution

Large language models dissolve the bottleneck. They serve as characteristica generators:

For NLP, the LLM proposes logical forms for natural language sentences. For software, the LLM proposes formal specifications for architectural constraints. For verification, the LLM proposes test cases, invariants, pre- and postconditions.

The typed slot grammar structures the proposals into well-formed characteristica. The calculus checks them against the grounding. The human reviews. The loop is: generate → ground → verify → repair.

9.3 The Bitter Lesson, Revisited

Sutton's "bitter lesson" (2019) argues that general methods leveraging computation consistently defeat methods leveraging human knowledge. The formal NLP pipeline we describe sits squarely on the side Sutton warns against.

We agree with Sutton's historical analysis. The representation was never the problem. The annotation was. LLMs eliminate the annotation bottleneck, making both sides scale with computation: the LLM generates the characteristica (scaling with model size and training data), and the calculus checks it (scaling with graph size and iteration count). What no longer scales with human effort is the construction of the formal infrastructure, because the LLM replaces the human in that role.

Our position is not that the bitter lesson was wrong. Our position is that LLMs are the mechanism by which its prescription—scale with compute—finally becomes applicable to formal semantics.

10 The Transformer Mapping

10.1 A Fourth Line of Evidence

We have argued for the universality of the characteristica on three grounds: historical convergence, parsimony, and completeness. We now present a fourth: the transformer architecture—the most successful neural architecture for language—independently converges on the same computational structure.

This is not a proof. It is a structural correspondence that we find suggestive and worth stating explicitly.

10.2 The Mapping

Attention as predicate binding. The attention mechanism maps a query to a set of key-value pairs and produces a weighted combination. The characteristica's predicates map a function name to a set of role-labeled arguments. Both are key-value lookups over structured representations. The attention head selects which entities fill which roles; the predicate's role labels specify the same binding explicitly.

Feedforward layers as knowledge base lookup. The feedforward network in each transformer layer has been shown to store factual associations—entity-attribute pairs retrievable by pattern matching. This is functionally a knowledge base: given an entity, retrieve its properties. The characteristica's ground facts serve the same role explicitly.

Layers as proof depth. Each transformer layer applies one step of computation. Stacking layers enables multi-step reasoning. In the characteristica, each step of belief propagation applies one round of inference rules. Depth of inference corresponds to number of layers or iterations.

Softmax as probabilistic grounding. The softmax function converts logits to probability distributions. The characteristica's modal quantifiers (*always*, *usually*, *sometimes*) convert logical rules to probabilistic weights. Both map discrete structure to degrees of belief.

10.3 The Implication

If the transformer has learned, through gradient descent on language data, to implement something functionally equivalent to typed predicate binding, knowledge base lookup, multi-step inference, and probabilistic grounding—then the characteristica is not merely a useful formalism imposed from outside. It is the structure that language itself requires, rediscovered independently by both

the mathematical tradition (Frege through Prawitz) and the statistical tradition (word2vec through GPT).

This convergence from radically different starting points—axiomatic logic and stochastic gradient descent—is perhaps the strongest evidence that the characteristica is not arbitrary but necessary.

11 Software Engineering

11.1 One Language, Two Domains

"All men are mortal" and "all requests require authentication" have the same logical form:

```
always [x:e]: man(theme: x) -> mortal(theme: x)
always [x:e]: request(theme: x) -> requires_auth(theme: x)
```

The characteristica does not care whether the entities are people or HTTP requests. Typed predicates with quantified Horn clauses verify natural language reasoning and software architecture constraints with the same machinery. One formal language, two domains.

11.2 The Next Bottleneck

LLM-assisted software development is compressing the time from specification to code toward zero. But once that time hits zero, the bottleneck shifts. The question is no longer "how do I turn this spec into code faster?" but "how do I know the code is correct?"

Testing catches some of this. But testing is sampling—it checks specific cases, not general properties. "All requests require authentication" is a universal claim. No finite set of tests can verify a universal claim. Logic can.

11.3 Formal Architecture as Backbone

Write the high-level structure of the application in the characteristica. Not the implementation—the architecture. The invariants, the contracts, the constraints that must hold across the entire system:

```
always [x:e]: request(theme: x) -> requires_auth(theme: x)
always [x:e]: write_op(theme: x) -> has_transaction(theme: x)
always [x:e]: user_data(theme: x) -> encrypted(theme: x)
always [x:e]: api_endpoint(theme: x) -> rate_limited(theme: x)
```

These compile. They run inference. They can be checked by the calculus for consistency, completeness, and entailment. The architecture becomes a logic program—a formal specification that both human and AI can read, that the calculus can verify, and that serves as the backbone against which implementations are checked.

11.4 Why the Characteristica

Existing formal verification tools (TLA+, Alloy, Z) can specify software architecture. But they suffer from the same annotation bottleneck. The characteristica has two advantages: proximity to natural language (the spec reads like English with types, and an LLM can generate it) and a shared language with reasoning (the same calculus, the same proof trees, the same RLVF training loop). Software verification is not a separate tool—it is an application of the universal language.

12 Experiments: Eating Our Own Dogfood

12.1 Method

The standard experiment for a new system is to run it on benchmark data. Our system's contribution is not new inference machinery or new parsing results—those were reported in Coppola (2024, 2026). The contribution of this paper is the thesis that the typed logical language is a universal language for reasoning.

The appropriate experiment for a universality claim is to test expressiveness: can the characteristica express what we need it to express? We test this by formalizing the arguments of this paper—the thesis itself—in the characteristica, and verifying them with the calculus. This is "eating our own dogfood": using the system to verify the claims made about it.

12.2 The Fixed Point

The convergence argument—that Aristotle and Pāṇini independently arrived at typed predicate structure, therefore the structure is universal—is a chain of typed predicates with quantified Horn clauses:

```
arrived_at(agent: aristotle, result: typed_structure)
arrived_at(agent: panini, result: typed_structure)
independent_of(agent: panini, reference: aristotle)
always: arrived_at(agent: x, result: typed_structure)
      & arrived_at(agent: y, result: typed_structure)
      & independent_of(agent: x, reference: y)
      -> universal_structure(theme: typed_structure)
? universal_structure(theme: typed_structure)
```

This compiles. The calculus runs inference. The answer is *yes*. The argument for universality is verified by the system whose universality is being argued.

This is a fixed point: the language is a fixed point of its own expressiveness claims. We claim it can express any natural language reasoning. It can express that claim. It can verify it.

12.3 Coverage

We have not encountered a barrier to expressing new ideas in the characteristica. Every argument we have wanted to formalize—philosophical (convergence, parsimony), practical (verification, trustworthiness), strategic, social—has compiled and verified without requiring extensions to the language or the calculus.

Beyond the thesis arguments, the same characteristica handles:

Classical reasoning: "All men are mortal. Socrates is a man." (Aristotle's syllogism)

Strategic reasoning: "Never let anyone know what you are thinking" — modus ponens with negation.

Social reasoning: "Friends must be treated as potential enemies" — conjunction with multi-role predicates.

Software constraints: "All requests require authentication" — quantified architectural invariants.

The language does not change across domains. The predicates change. The logic is the same. This is the universality claim in practice.

12.4 Prior Empirical Results

The inference engine and grammar were evaluated in Coppola (2026): 44/44 inference test cases across 22 reasoning patterns, 33/33 sentences parsed with zero ambiguity, 95% PP attachment accuracy. We do not re-report these results but note that they establish the mechanical correctness of the system on which the present experiments depend.

12.5 Limitations

The examples are human-considered, not large-scale. Each was selected because we found it interesting or important enough to formalize. We do not report large numbers. The point is not volume but the absence of a barrier: everything we have wanted to express, we have expressed. The barrier we expected to find has not appeared. If it did, the response would be to extend the lexicon—add new predicates, entities, rules—while the logical machinery remains unchanged. This is what the sufficiency thesis predicts: the remaining work is lexical, not logical.

Full examples are available at `https://github.com/gregorycoppola/world`.

13 Related Work

13.1 Probabilistic Logic

Markov Logic Networks (Richardson and Domingos, 2006) combine first-order logic with Markov random fields, weighting logical formulas and performing inference via MCMC sampling. The LBN restricts expressiveness to the forward fragment but gains efficient inference via belief propagation rather than MCMC. ProbLog (De Raedt, Kimmig, and Toivonen, 2007) extends Prolog with probabilistic facts and uses exact inference via weighted model counting. The LBN trades exactness for efficiency through approximate belief propagation and uses role-based predicates rather than positional arguments. Neither MLNs nor ProbLog address parsing natural language to logical form.

13.2 Neuro-Symbolic Systems

Garcez, Lamb, and Gabbay (2009) proposed encoding logical rules in neural network weights. Manhaeve et al. (2018) extended ProbLog with neural predicates trained end-to-end. Rocktäschel and Riedel (2017) introduced differentiable theorem provers. These approaches seek to make logic differentiable for end-to-end training. Our architecture makes a different choice: a clean separation between the neural component (the LLM) and the symbolic component (the grammar and LBN). This separation means we cannot train end-to-end, but the reasoning component is exact—conclusions follow from inspectable derivations, not differentiable approximations.

13.3 Proof Assistants

Lean and Coq provide formal verification for mathematics. They satisfy convergence, parsimony, and completeness for mathematical proof. The characteristica differs in operating over natural language semantics rather than mathematical types. "Jack trusts Jill" and "all requests require authentication" are first-class propositions in the characteristica but awkward to express in Lean's dependent type theory. The characteristica is arguably the simplest verification language for natural language reasoning, which makes it the most learnable—and learnability is what matters for RLVF.

13.4 Semantic Parsing

The compositional semantic parsing tradition, from Montague (1973) through Zettlemoyer and Collins (2005) to modern neural semantic parsers, addresses the same text-to-logic problem. Our typed slot grammar is deliberately simple, trading coverage for exactness. The LLM preprocessing layer extends coverage without sacrificing the guarantee of zero ambiguity on handled constructions.

13.5 LLMs and Formal Reasoning

Recent work has explored LLMs as reasoners, including chain-of-thought prompting, self-consistency, and tree-of-thought approaches. These methods improve the appearance of reasoning but do not provide verification—the output remains ungrounded token sequences. Our approach is complementary: the LLM generates candidate reasoning, and the characteristica and calculus verify it.

13.6 Leibniz and the Universal Language

The idea of a universal formal language dates to Leibniz (1685) and has been pursued through Frege's Begriffsschrift (1879), Russell and Whitehead's Principia Mathematica (1910), and the logical positivist program. These efforts focused on mathematical foundations. Our contribution is connecting this tradition to modern AI through probabilistic graphical models and LLM-assisted construction, and arguing that the resulting system has immediate practical consequences for hallucination, interpretability, RLVF, and alignment.

14 Conclusion

We have argued that the typed logical language developed across this series of papers—entities with types, predicates with labeled roles, quantified Horn clauses with modal weights, negation, propositions as arguments, and lambda abstraction—is a realization of Leibniz's characteristica universalis: a universal formal language sufficient for expressing any proposition expressible in natural language. And that the Logical Bayesian Network—belief propagation over AND/OR/NEG factor graphs—is a realization of his calculus ratiocinator: a mechanical procedure for reasoning over that language, extended from certainty to degrees of belief.

This thesis yields four consequences. Hallucination is structurally impossible because ungrounded conclusions cannot be derived. Interpretability is free because the factor graph is the explanation. RLVF becomes possible because the calculus is a mechanical verifier for reasoning. Alignment reduces to verification because intent and output share the same formal language.

The characteristica is not just for verification—it is the common interface between human and machine. If both parties reason in the characteristica (humans implicitly, machines explicitly), then learning to think more explicitly in typed predicates and rules is not learning a foreign language. It is becoming conscious of the language you already think in. Communication improves because translation shrinks. This is what "universal grammar" means in practice: not a biological claim, but a shared formal language that both humans and machines can read, write, and verify.

The immediate next steps are extending grammar coverage to the full set of reasoning patterns, scaling to larger knowledge bases, learning rule weights from data via the EM algorithm, and demonstrating the RLVF training loop. The logical and inferential machinery is in place. What remains is lexical—determining which words map to which predicates and types, and demonstrating this at scale. The bitter lesson tells us this is precisely the kind of work that should be done by computation, not by hand. We agree, and the LLM is the computation.

All code, test data, and documentation are open source at `https://github.com/gregorycoppola/world`.

Acknowledgments

This work was developed with substantial assistance from large language models, a fact we acknowledge both for transparency and as evidence for the paper's central argument that LLMs can serve as annotators and collaborators in formal research.

The 2024 paper and initial codebase were developed primarily with ChatGPT (OpenAI). The current implementation, coverage tests, grammar rules, and this paper were developed primarily with Claude (Anthropic). Both systems contributed to code development, test generation, grammar authoring, and paper drafting. The fact that two independent LLM systems could each engage productively with the formal representations described here is itself evidence that the representations are well-defined and machine-legible.

References

[1] Aristotle. *Prior Analytics*, c. 350 BC.

[2] Pāṇini. *Aṣṭādhyāyī*, c. 400 BC.

[3] Leibniz, G.W. *The Art of Discovery*, 1685.

[4] Frege, G. *Begriffsschrift*, 1879.

[5] Gentzen, G. "Untersuchungen über das logische Schließen." *Mathematische Zeitschrift*, 39:176–210, 405–431, 1934.

[6] Prawitz, D. *Natural Deduction: A Proof-Theoretical Study*. Almqvist & Wiksell, 1965.

[7] Montague, R. "The Proper Treatment of Quantification in Ordinary English." In *Approaches to Natural Language*, pp. 221–242, 1973.

[8] Pearl, J. *Probabilistic Reasoning in Intelligent Systems*. Morgan Kaufmann, 1988.

[9] Steedman, M. *Surface Structure and Interpretation*. MIT Press, 1996.

[10] Richardson, M. and Domingos, P. "Markov Logic Networks." *Machine Learning*, 62:107–136, 2006.

[11] De Raedt, L., Kimmig, A., and Toivonen, H. "ProbLog: A Probabilistic Prolog and Its Application in Link Discovery." In *Proceedings of IJCAI*, 2007.

[12] Garcez, A.S., Lamb, L.C., and Gabbay, D.M. *Neural-Symbolic Cognitive Reasoning*. Springer, 2009.

[13] Kahneman, D. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.

[14] Rocktäschel, T. and Riedel, S. "End-to-End Differentiable Proving." In *Proceedings of NeurIPS*, 2017.

[15] Manhaeve, R., Dumancic, S., Kimmig, A., Demeester, T., and De Raedt, L. "DeepProbLog: Neural Probabilistic Logic Programming." In *Proceedings of NeurIPS*, 2018.

[16] Sutton, R. "The Bitter Lesson." Blog post, March 13, 2019.

[17] Coppola, G. "The Quantified Boolean Bayesian Network: Theory and Experiments with a Logical Graphical Model." arXiv:2402.06557, 2024.

[18] Karpathy, A. "Vibe Coding." Post on X, February 2, 2025.

[19] Coppola, G. "Statistical Parsing for Logical Information Retrieval." 2026.

[20] Zettlemoyer, L.S. and Collins, M. "Learning to Map Sentences to Logical Form." In *Proceedings of UAI*, pp. 658–666, 2005.

[21] Cooper, G.F. "The Computational Complexity of Probabilistic Inference Using Bayesian Belief Networks." *Artificial Intelligence*, 42(2–3):393–405, 1990.

[22] Roth, D. "On the Hardness of Approximate Reasoning." *Artificial Intelligence*, 82:273–302, 1996.

[23] Koller, D. and Friedman, N. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, 2009.

[24] Neapolitan, R.E. *Learning Bayesian Networks*. Prentice Hall, 2003.

[25] Murphy, K., Weiss, Y., and Jordan, M.I. "Loopy Belief Propagation for Approximate Inference: An Empirical Study." In *Proceedings of UAI*, pp. 467–476, 1999.

[26] Chomsky, N. *Syntactic Structures*. Mouton, 1957.

[27] Chomsky, N. *Aspects of the Theory of Syntax*. MIT Press, 1965.

[28] Bar-Hillel, Y. "A Quasi-Arithmetical Notation for Syntactic Description." *Language*, 29(1):47–58, 1953.

[29] Montague, R. "Universal Grammar." *Theoria*, 36:373–398, 1970.